

Case 13: Negative — Invalid Data Types in Payload

Intermediate+

Verify the API rejects payloads where field values have wrong data types.

New concept: Testing wrong data types in request body

API Endpoint

POST `https://jsonplaceholder.typicode.com/posts`

What You Will Learn

- Data type validation in APIs
- Sending wrong types: string where int expected, int where string expected
- 400 Bad Request for invalid data
- Parametrizing multiple invalid payloads

Study Plan

Step 1 — What is type validation?

APIs expect specific types for each field. `userId` should be an integer, `title` should be a string. Sending `userId='abc'` or `title=12345` should be rejected with 400.

Step 2 — Build wrong-type payloads

Create payloads with intentionally wrong types: `{'title': 123, 'body': 'text', 'userId': 1}` — `title` is an int not a string. `{'title': 'text', 'body': 'text', 'userId': 'abc'}` — `userId` is a string not an int.

Step 3 — Parametrize the bad payloads

Think: what is the changing value between each test run? Each parametrize entry is one bad payload dict. How many entries will your parametrize list have?

Step 4 — What to assert

A strict API returns 400. JSONPlaceholder is lenient — it accepts wrong types. Use `xfail` to mark the expected 400 assertion.

Step 5 — Think about the loop

After getting the response, do you need a for loop to validate multiple fields? Or is checking the status code enough for a negative test?

Think Before You Code

Ask yourself these questions before writing a single line:

- What is the changing value between each test run — the URL or the payload?
- What type should `userId` be — a string or an integer?
- After getting the response, do you need a for loop, or is one assert enough?
- How many parametrize entries do you need — one per bad payload?
- What would a real API return when it receives the wrong data type?

Your Task

Write a parametrized pytest test that sends POST requests with wrong data types.

Test these invalid payloads:

1. {'title': 12345, 'body': 'text', 'userId': 1} (title is int, not string)
2. {'title': 'text', 'body': 'text', 'userId': 'abc'} (userId is string, not int)
3. {'title': True, 'body': False, 'userId': 1} (title/body are booleans)

Before writing:

- What is the parametrize variable?
- Do you need a for loop inside the test?
- How do you handle JSONPlaceholder's leniency?

The test must verify:

1. Parametrize all 3 invalid payloads
2. Mark as xfail — JSONPlaceholder won't reject wrong types
3. Assert status code is 400

BONUS: Print `response.json()` for each — what does JSONPlaceholder echo back when you send wrong types? Does it convert them or return them as-is?

Hint: Parametrize variable is the payload dict. Each entry in the list is one bad payload. No loop needed inside the test.